STUDENT GUIDE

Week 9 | Code Review 1

Step-by-step practical guide | Spring 2026 LocalLoop case study

Contents

1 What are you being assessed on? 2

2 Preparation before Week 9 2

3 Step-by-step: what the code author must do 3

4 Step-by-step: what the peer reviewer must do 3

5 Submit your main-branch Git history 4

6 What to show during the practical 4

7 Understand the actual LocalLoop problem 5

8 Student activity 1: turn the case into requirements 6

9 Student activity 2: design a small, reviewable feature 7

10 Student activity 3: example technical design (optional) 8

11 Student activity 4: illustrative REST API contract 9

12 Student activity 5: implement and test your contribution 10

13 Student activity 6: guided peer-review exercise 11

14 Final practical and LMS submission checklist 12

15 Sources and what is official 12

Read first. This guide uses the attached official Spring 2026 case study, **LocalLoop Community Food Rescue Platform**, alongside the Assessment 2 Code Reviews brief. The case-study document gives a domain, seven conceptual business functions, and a requirement to develop at least four core functions. It **does not prescribe specific endpoints, database tables, technology stacks, or Week 9 deliverables**. Such details below are **suggested implementation and practice examples**, not additional

official requirements. The Learning Guide rubric and live vUWS instructions remain authoritative.

1 What are you being assessed on?

Assessment 2: Code Reviews accounts for 20% of the unit. Practical Assessment 1 takes place in Week 9 (10%); Practical Assessment 2 takes place in Week 12 (10%). Although you work in a group repository, the code review assessment is individual.

You must do	Evidence to show
Write and test your own code	Your own source-code contributions and meaningful unit/integration tests.
Merge your contribution into main	Git history on the shared group repository, with your code traceable.
Review teammates' code	Useful GitHub Pull Request reviews/comments, including testing evidence.
Submit before your registered practical	Your student-ID-named main-branch commit-history CSV to vUWS.
Attend and explain your work	Live discussion/demo with the tutor during your registered practical.

Important official consequences. No successfully committed code on main means zero for the entire code-review assessment under the brief. Missing your registered practical means zero for that practical, irrespective of GitHub activity or LMS submission. Do not delete PRs you want assessed.

2 Preparation before Week 9

1. **Confirm your practical.** Check the date, time, and location of your registered practical; the CSV must be submitted before that session begins.
2. **Locate the correct repository.** Access your group's GitHub Group Assignment repository. Confirm you can see the main branch, your team members and existing Pull Requests.
3. **Understand the actual project.** Read the Spring 2026 **LocalLoop Community Food Rescue Platform** case study and the team's Software Design Document, agreed scope, task board, coding conventions and tests.
4. **Coordinate responsibilities.** Allocate a small, meaningful change to each student; agree which teammate will review each change. A PR should contain newly written or substantially changed code.
5. **Prepare tools.** Install Git and your team's chosen runtime/dependencies (for example, Node.js); clone the group repository; check that the application and its existing tests run before making changes.

3 Step-by-step: what the code author must do

1. **Synchronise main.** From the repository folder:

```
git switch main
git pull origin main
git status
```

2. **Create a feature branch.** Use a descriptive branch name for your own new feature or fix. Do not develop the feature directly on main.

```
git switch -c feature/food-listing-validation
```

3. **Implement a focused change.** Write code relevant to the group project. Follow agreed naming, structure, input validation, error handling and security conventions. Keep unrelated refactors out of the PR.
4. **Add meaningful tests.** Include successful behaviour plus negative/boundary cases; run existing and new tests locally. Record how the change was tested.

```
npm install   # if the team uses Node.js
npm test      # if a test script is configured
```

5. **Commit and push your changes.** Use small, descriptive commits so others can trace the work. Example:

```
git add .
git commit -m "Validate LocalLoop food listing fields"
git push -u origin feature/food-listing-validation
```

6. **Open a Pull Request to main.** In GitHub, use base: main and compare: your feature branch. Write a description: problem, changes, test commands/results, risks, and reviewer request.
7. **Respond to feedback.** Read reviewer comments, discuss unclear issues, fix genuine problems and add commits to the same PR branch. Re-run your tests.
8. **Merge after review.** Follow the group's repository review policy. Confirm your code actually appears on main. A PR that exists but has not contributed your code to main does not satisfy the author requirement.

```
git switch main
git pull origin main
git log --oneline -n 10
```

4 Step-by-step: what the peer reviewer must do

1. **Select an eligible PR.** At least 48 hours before your practical, check open PRs as directed in the official checklist. Review someone else's new or substantially changed code that

has not already been reviewed for this assessment.

2. **Understand the request.** Read the PR description, inspect the diff and ask the author about the intended behaviour and its role in the application.
3. **Run the change locally.** Fetch or check out the PR branch, install dependencies, run its tests and manually exercise key endpoints. Record exact outcomes.
4. **Inspect six code-quality dimensions.** Check functionality, coding conventions, readability, performance, security and error handling; also assess whether tests genuinely exercise the feature.
5. **Leave actionable GitHub comments.** Identify file/line, explain the risk, give a specific recommendation, and mention relevant tests. Note effective design choices too.
6. **Complete the review.** Use GitHub's review action (comment, request changes or approve as appropriate). Make sure the PR and comments remain accessible for your tutor.

> **Review scope.** Review the teammate's authored changes, not framework/library internals. The same unchanged code should not be counted repeatedly as a new review.

5 Submit your main-branch Git history

Before your registered Week 9 practical, open Git Bash in your group repository. Switch to and update main. The assignment appendix provides this exact export command (replace the filename placeholder with your student ID):

```
git switch main
git pull origin main
git log --pretty=format:'"%h","%an","%aD","%s",' > <studentid>-history.csv
```

Example for a fictitious ID: 12345678-history.csv. Open the generated file and check that commits are present and that your own contribution can be located. The file lists the repository history, not just your own commits.

Submit the CSV at the vUWS link titled "SUBMIT COMMIT HISTORY" before the start of your registered practical. Keep the GitHub repository and relevant Pull Requests available to demonstrate live. The CSV does not replace merging your code or conducting reviews.

6 What to show during the practical

- Your GitHub repository on main, with the feature you contributed and the commit/PR that introduced it.
- The relevant source file(s): what they do, design decisions, error handling and security safeguards.
- At least one meaningful automated test and an explanation of what would cause it to fail.
- Your teammate's PR and your review: line-specific comments, practical testing, and resulting changes.
- Your submitted CSV and confirmation it was uploaded before your registered practical.

OFFICIAL SPRING 2026 CASE STUDY

LocalLoop Community Food Rescue Platform

> **Official case-study facts (Spring 2026).** A local council has commissioned a platform connecting food retailers, cafés, restaurants, community organisations, volunteers and residents to redistribute surplus food. The team must analyse functional, non-functional and system requirements, design an **Event-Driven Architecture**, and develop **at least four core business functions** over the semester. The seven functions below are **proposed areas**, not a fixed API specification.

7 Understand the actual LocalLoop problem

7.1 The business problem

Businesses have safe-to-consume surplus food, but redistribution currently depends on calls, emails, spreadsheets and social media. Information may be old or incomplete, collection deadlines are missed, reservations are duplicated, and food is wasted. LocalLoop should allow businesses to announce available food and eligible community users/organisations to locate, reserve and collect it. This summary follows the supplied two-page Spring 2026 case study.

7.2 The seven business functions in the official case study

Function (source)	What the development team must analyse
1. User and Organisation Management	Accounts and permissions for individuals, businesses, community organisations, volunteers and council administrators.
2. Surplus Food Listing Management	Publish food category, quantity, collection location, storage requirements, allergen information and collection deadline.
3. Search and Discovery	Filter food by location, category, quantity, dietary suitability, collection time and other relevant attributes.
4. Reservation and Collection Management	Eligible users reserve food, coordinate pickup, and progress a reservation from requested to collected or cancelled.
5. Food Safety and Compliance Management	Require safety information; administrators review listings, record issues and remove unsuitable listings.
6. Notification and Reminder System	Communicate requests, approvals, approaching deadlines, cancellations, instruction changes and suitable new food.

Function (source)	What the development team must analyse
7. Impact Tracking and Reporting	Report rescued food, completed collections, participating organisations and estimated waste reduction.

Critical scope distinction. The source explicitly says **minimum four core business functions**. Your team chooses and justifies at least four, then derives details such as validation rules, roles, APIs, database entities, monitoring and tests. The example requirements and API design in the following sections illustrate one possible solution; they are **not stated in the source case study**.

8 Student activity 1: turn the case into requirements

What to produce: a short requirements table (for example, in your Software Design Document or issue board), with a unique ID, source business function, user story, acceptance condition and priority.

1. Read the two-page LocalLoop case study. Underline the actors, the problem and the seven functions.
2. As a team, select at least four core functions and briefly justify why they represent a useful working system. A possible set is user/organisation management, food listings, discovery, and reservation/collection.
3. Agree what constitutes an *eligible* collector, a valid collection deadline, an available listing, a successful collection and an acceptable food safety record. These points are open design decisions.
4. Write testable functional requirements and relevant non-functional requirements (privacy, security, availability, response time, data integrity, accessibility and monitoring).
5. Record assumptions instead of silently presenting them as facts from the official case study.

ID	Illustrative requirement	Observable acceptance condition
FR-01	Business publishes surplus food	Required category, quantity, location, storage, allergen and deadline fields are validated; an eligible business can create the listing.
FR-02	User searches available listings	Search returns only discoverable, currently available listings matching selected criteria.
FR-03	Approved collector requests reservation	A permitted collector can request a valid quantity before the deadline; available quantity cannot become negative.
FR-04	Collector confirms pickup	An authorised actor can mark the reservation collected; the change is recorded.

ID	Illustrative requirement	Observable acceptance condition
NFR-01	Protect account and organisation data	Unauthenticated/unauthorised requests cannot modify other organisations' listings.
NFR-02	Preserve booking consistency	Two simultaneous requests must not reserve more food than the available amount.

8.1 Worked user story

As an approved community organisation member, I want to request two portions of a published food listing so that our volunteers can collect them before the deadline.

Suggested acceptance tests: (1) two portions are available and the deadline is future: request recorded; (2) zero/negative quantity: rejected; (3) no eligible membership: denied; (4) expired listing: rejected; (5) concurrent requests: total reserved never exceeds the available quantity. The team's actual reservation policy must define whether a request immediately allocates food or awaits approval.

9 Student activity 2: design a small, reviewable feature

9.1 Choose a suitable Week 9 contribution

Review the project's current progress. Pick a small feature or bug fix that is not a repeat of already reviewed work. Good possibilities are listing-field validation, secure listing creation, filtering by category/deadline, reservation state checks, food safety metadata validation, or a notification event handler. **These are suggested PR topics, not a separate Week 9 minimum set.**

9.2 Suggested responsibilities for a four-person team

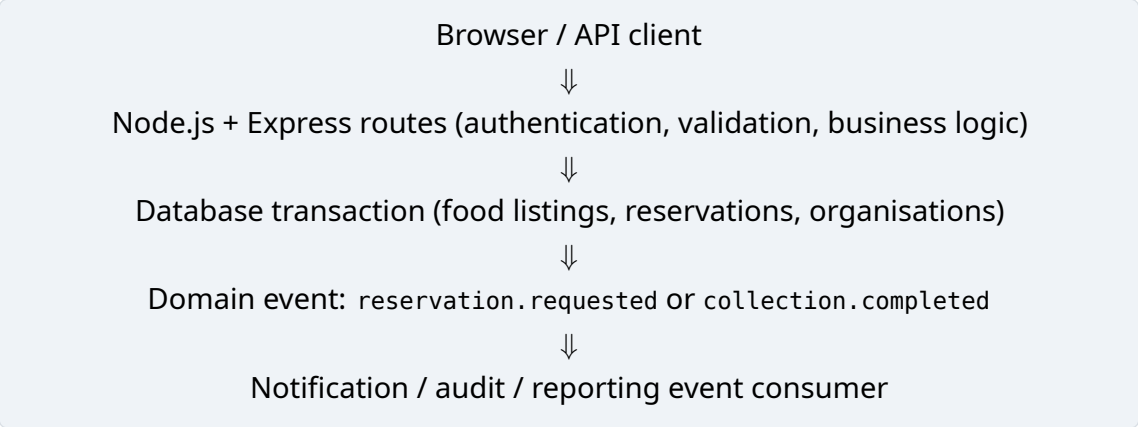
Example PR	Code and tests to contribute	What the peer reviewer checks
A – Listing validation	Create or update a food listing; validate quantity, allergens, storage information and deadline.	Invalid inputs, business ownership and safe errors.
B – Discovery	Search listings by category and deadline, with optional location filters.	Correct filters, visibility, pagination and tests.
C – Reservation	Request a number of portions, reject expired/unavailable food, prevent over-allocation.	Race conditions, permissions, transaction safety and boundary tests.

Example PR	Code and tests to contribute	What the peer reviewer checks
D – Collection events	Publish or handle `reservation.requested` / `collection.completed` events; log outcomes.	Event payloads, duplicate delivery and error handling.

Before coding: open a tracking issue, identify the files or module you will own, agree with a reviewer what constitutes "done", and write two or more test scenarios. Each team member should both contribute code that reaches `main` and meaningfully review a teammate's PR.

10 Student activity 3: example technical design (optional)

The source requires an Event-Driven Architecture but does not dictate an implementation language or data store. For a JavaScript/Node.js team, the following is a **reasonable illustration**, not a mandatory stack.



Illustrative table	Possible fields
`users`	ID, email, password hash, user role, organisation ID, account status.
`organisations`	ID, name, type (business / community organisation), approval status.
`food_listings`	ID, business ID, category, quantity total, quantity available, unit, location, storage notes, allergens, deadline, listing status.
`reservations`	ID, listing ID, collecting user/organisation, quantity, requested time, status, collection time.
`domain_events`	Event ID, event type, resource ID, event time, safe payload, processed state.

Suggested design decisions to document: whether quantities represent portions or kilograms; whether approval is required before allocation; which roles can confirm collection;

whether cancelled quantities are released; how deadlines and time zones are represented; and what happens when a food safety administrator removes a listing. Use a database constraint/transaction strategy that prevents double allocation.

11 Student activity 4: illustrative REST API contract

> **Examples only.** The official LocalLoop source names business functions, not HTTP endpoints or status codes. If your team uses REST, the following endpoints can serve as a concrete starting point for implementation and tests.

Possible endpoint	Expected behaviour for one proposed design
`POST /api/auth/login`	Authenticate a user; reject invalid credentials; never return secrets.
`POST /api/food-listings`	An authorised business creates a listing; validate mandatory details; 201 if created.
`GET /api/food-listings`	Search/filter available listings; 200 with matching results, including collection deadline.
`GET /api/food-listings/:id`	Return discoverable listing details; 404 if missing or not publicly visible.
`POST /api/food-listings/:id/reservations`	Eligible collector requests a quantity; validate expiry, availability and permission; 201 if accepted, 409 for conflicting state.
`PATCH /api/reservations/:id`	Permitted actor updates state (e.g. approved, collected, cancelled) under a documented transition policy.
`POST /api/listings/:id/compliance`	A council administrator records compliance findings / takes authorised action.
`GET /api/reports/impact`	An authorised viewer sees aggregate completed collections and food rescued.

11.1 Example request and response

```
POST /api/food-listings/42/reservations
Authorization: Bearer <valid-access-token>
Content-Type: application/json

{ "quantity": 2 }

// Example: HTTP 201 (when the chosen policy accepts requests)
{ "reservationId": 105, "listingId": 42,
  "quantity": 2, "status": "requested" }
```

12 Student activity 5: implement and test your contribution

1. **Create a branch:** begin from a clean, updated `main`; use a descriptive feature-branch name (see the example in Section 3).
2. **Code one responsibility:** add validation and business logic, and ensure your role/ownership checks happen server-side.
3. **Write automated tests:** cover one successful path, one invalid input, one permission failure and any relevant boundary or concurrency case.
4. **Run and fix tests:** for Node.js projects, run `npm test`; inspect failures rather than simply reporting that tests exist.
5. **Commit and push:** create a focused commit and a PR targeting `main`; describe behaviour, tests and known limitations.
6. **Request peer review:** arrange a teammate's independent run of your code and respond to comments; fix defects and re-run the relevant tests.
7. **Merge and verify:** after the team has followed its review policy, verify that your contribution is visible on `main` and remains traceable.

12.1 Illustrative acceptance test scenarios

Scenario	Expected observable result
Business submits valid food listing	Saved with category, quantity, allergen, storage, location and collection deadline details.
Business omits allergen disclosure field	Validation fails under the team's defined mandatory-field policy.
User searches category "bakery"	Matching discoverable listings returned; unavailable/removed listings excluded.
Collector requests more food than is available	Request rejected without increasing allocated quantity.
Two collectors request the last portion simultaneously	No more than one portion is allocated; database quantity remains valid.
Collector tries to reserve food after deadline	Request rejected with a clear application-level error.
Unapproved actor edits another business's listing	Request denied; listing data unchanged.
Reservation successfully becomes "collected"	Persisted status transition and relevant domain event / audit entry.
Malformed request or database error	Safe response; no credentials or internal stack trace leaked.

12.2 Illustrative Jest/Supertest test structure

This is a sketch to adapt to the team's actual application, authentication and reservation rules. It is not a complete runnable test suite without a configured server and test database.

```
test('cannot reserve after collection deadline', async () => {
  const listing = await seedListing({
    quantityAvailable: 2,
    deadline: '2026-01-01T00:00:00Z'
  });
  const response = await request(app)
    .post(`/api/food-listings/${listing.id}/reservations`)
    .set('Authorization', `Bearer ${approvedCollectorToken}`)
    .send({ quantity: 1 });
  expect(response.status).toBe(409); // proposed API design
  expect(await availableQuantity(listing.id)).toBe(2);
});
```

13 Student activity 6: guided peer-review exercise

> **Exercise, not reference implementation.** The following **deliberately defective** pseudocode-like Express handler is for identifying risks. It makes assumptions about database helpers and authentication and must not be copied into a deployed system.

```
app.post('/api/food-listings/:id/reservations', async (req, res) => {
  const userId = req.body.userId;
  const qty = req.body.quantity;
  const listing = await db.get(
    `SELECT * FROM food_listings WHERE id = ${req.params.id}`
  );
  const available = listing.quantity_available;
  if (available >= qty) {
    await db.run(
      `INSERT INTO reservations (listing_id, user_id, quantity)
       VALUES (${req.params.id}, '${userId}', ${qty})`
    );
    await db.run(
      `UPDATE food_listings
       SET quantity_available = ${available - qty}
       WHERE id = ${req.params.id}`
    );
    return res.status(200).json({ status: 'reserved' });
  }
  res.send('Unavailable');
});
```

Student task. Identify **at least five** issues and submit them as review practice comments or discuss them with your tutor. For each: state the relevant line/operation, explain the impact, propose a correction and name a test that would expose the defect.

Review lens	Questions students should ask
Security	Can users impersonate others? Can interpolated query values be exploited? Is permission checked?
Food safety	Is the listing published, compliant, and still within its collection deadline?
Correctness	What happens for quantity 0, negative, decimal, or non-numeric input?
Concurrency	Can two requests reserve the same final portion? Are insert and quantity decrement atomic?
Error handling	What if the listing is missing or the database fails? Are HTTP status codes consistent?
Events and testing	Should a reservation event be emitted? Which positive, boundary and failure tests are missing?

13.1 Example useful reviewer comment

> **Reservation oversubscription:** The handler reads `quantity_available` before writing, without a transaction or conditional update. Two simultaneous requests can both see one remaining portion and each insert a reservation. Please use an atomic database operation and test two concurrent requests against a listing with one portion; assert that the final allocated total never exceeds one.

14 Final practical and LMS submission checklist

- ☐ Our team has agreed which **at least four** LocalLoop core functions it is implementing and recorded its design assumptions.
- ☐ I can explain the LocalLoop feature I personally implemented and show it running.
- ☐ My own code and tests are merged into the group's `main` branch.
- ☐ I have reviewed a teammate's new or substantially changed code in a GitHub Pull Request.
- ☐ I can show which tests I ran and the review comments I posted.
- ☐ I retained the PRs that will be assessed, including closed/unmerged PRs.
- ☐ I exported the Git history from `main` and checked the generated CSV.
- ☐ I submitted `<studentid>-history.csv` to vUWS **before** my registered practical.
- ☐ I will attend the registered practical and answer questions about my code and review.

15 Sources and what is official

- **Case study:** *COMP3028 – Software Construction, Spring 2026 Case Study: LocalLoop Community Food Rescue Platform*, pp. 1–2 (uploaded PDF). Authoritative for the food-rescue scenario, Event-Driven Architecture objective, conceptual functions and minimum four core functions.

- **Assessment:** *COMP3028 – Software Construction, Assessment 2: Code Reviews (20%)*, pp. 1–4 (uploaded PDF). Authoritative for PR review duties, `main`-branch contribution, CSV export, practical attendance, timing and stated consequences.
- **Learning Guide:** Contains the marking rubric according to the Assessment 2 handout; consult the current unit materials for the actual criteria. All endpoint designs, technical choices, example test cases and weekly PR allocations in this document are **teaching illustrations** derived from (not quoted from) the case study.

vUWS: Submit main-branch Git commit history before your registered practical.